Отчёт по практической работе

Тема: Интеграционные тесты в pytest

Студент: Новиков А А
Группа: ИСПП 2309(2)
Дата выполнения: 8 апреля 2026 г.

Что я сделал. пошаговое объяснение

Шаг 1. Создание структуры проекта

Я создал в VS Code следующую структуру папок и файлов:

```text

project/
├── app/
│   ├── repository.py
│   ├── api.py
│   └── files.py
├── tests/
│   ├── integration/
│   │   ├── test_repository.py
│   │   ├── test_api.py
│   │   └── test_files.py
│   ├── conftest.py
│   └── pytest.ini
```

Папка `integration` показывает, что внутри лежат интеграционные тесты.

Шаг 2. Создание кода приложения

Я скопировал предоставленный код в соответствующие файлы:

app/repository.py — работа с SQLite БД (добавление, поиск, получение пользователей)

app/api.py — Flask-приложение с двумя эндпоинтами (POST и GET /users)

app/files.py — работа с файловой системой (сохранение и чтение отчётов)

Шаг 3. Настройка pytest

Файл: pytest.ini

```
markers =
    integration: integration tests
```

Файл: conftest.py

```
import sqlite3
import pytest
from app.repository import init_db
from app.api import create_app, users


def db_connection():
    """Фикстура создаёт in-memory БД для каждого теста изолированно"""
    connection = sqlite3.connect(":memory:")

    init_db(connection)
    yield connection
    connection.close()



def client():
    """Фикстура создаёт тестовый клиент Flask"""
    users.clear()
    app = create_app()
    app.config["TESTING"] = True
    with app.test_client() as client:
        yield client
```

Объяснение фикстур:

Фикстура	Что делает	Зачем
`db_connection`	Создаёт SQLite БД в памяти	Каждый тест получает чистую БД, тесты не зависят друг от друга
`client`	Создаёт тестовый клиент Flask	Можно отправлять запросы без запуска сервера

Выполнение заданий

Задание 1. Интеграционный тест для базы данных

Файл: tests/integration/test_repository.py

```
import pytest
from app.repository import add_user, get_user_by_name


def test_add_and_get_user(db_connection):

    add_user(db_connection, "Алексей")


    user = get_user_by_name(db_connection, "Алексей")


    assert user is not None, "Пользователь не найден в БД"


    user_id, user_name = user
    assert user_name == "Алексей"
    assert user_id == 1
```

Результат выполнения:

```text
tests/integration/test_repository.py::test_add_and_get_user PASSED [100%]
```

Задание 2. Тест для списка пользователей

Добавил в тот же файл:

```python
def test_get_all_users(db_connection):

    add_user(db_connection, "Мария")
    add_user(db_connection, "Дмитрий")

    users = get_all_users(db_connection)

    assert len(users) == 2

    names = [name for _, name in users]
    assert "Мария" in names
    assert "Дмитрий" in names
```

Результат:

```text
tests/integration/test_repository.py::test_get_all_users PASSED [100%]
```

Задание 3. Интеграционный тест API

Файл: tests/integration/test_api.py

```python
import pytest
from app.api import users



def test_create_and_get_user_via_api(client):
    new_user = {"name": "Екатерина", "email": "katya@example.com"}
```

```
    post_response = client.post("/users", json=new_user)



    assert post_response.status_code == 201
    assert post_response.json["name"] == "Екатерина"



    get_response = client.get("/users")



    assert get_response.status_code == 200



    users_list = get_response.json
    assert len(users_list) == 1
    assert users_list[0]["name"] == "Екатерина"
```

Результат:

```text

tests/integration/test_api.py::test_create_and_get_user_via_api PASSED [100%]
```

Задание 4. Интеграционный тест файловой системы

Файл: tests/integration/test_files.py

```
import pytest
import tempfile
import os
from app.files import save_report, read_report

def test_save_and_read_report():

    with tempfile.NamedTemporaryFile(mode="w", suffix=".txt", delete=False) as tmp:
        temp_path = tmp.name

    try:
```

```
        test_text = "Это отчёт о тестировании интеграции"
        save_report(temp_path, test_text)



        read_text = read_report(temp_path)



        assert read_text == test_text
    finally:

        if os.path.exists(temp_path):
            os.remove(temp_path)
```

Результат:

```text

tests/integration/test_files.py::test_save_and_read_report PASSED [100%]
```

Задание 5. Параметризация интеграционного теста

Добавил в test_repository.py:

```
@pytest.mark.integration
@pytest.mark.parametrize("user_name", [
    "Анна",
    "Максим",
    "Ольга-Мария",
    "Jhon Doe",
    "Пользователь123"
])
def test_add_user_parametrized(db_connection, user_name):
    """Параметризованный тест — проверяет добавление разных имён"""
    add_user(db_connection, user_name)
    user = get_user_by_name(db_connection, user_name)

    assert user is not None
    assert user[1] == user_name
```

Результат:

```text
```

```
test_repository.py::test_add_user_parametrized[Анна] PASSED
test_repository.py::test_add_user_parametrized[Максим] PASSED
test_repository.py::test_add_user_parametrized[Ольга-Мария] PASSED
test_repository.py::test_add_user_parametrized[Jhon Doe] PASSED
test_repository.py::test_add_user_parametrized[Пользователь123] PASSED
```

Отличие интеграционных тестов от unit-тестов (вывод)

Критерий	Unit-тест	Интеграционный тест
Что тестирует	Один класс/функцию изолированно	Взаимодействие нескольких компонентов
Моки	Активно использует mock	Минимум моков, реальные зависимости
БД	Использует mock или in-memory	Реальная БД (in-memory для скорости)
Файлы	Не трогает диск	Создаёт временные файлы
Скорость	Очень быстрые	Медленнее, но дают уверенность
В моём коде	Нет моков вообще	БД, API, файлы работают по-настоящему